triad

Executive Summary

This module computes a reference attitude frame using the TRIAD method that aligns the spacecraft body thrust axis with a commanded inertial thrust direction, while using the Sun direction to resolve the remaining rotational degree of freedom. The TRIAD method constructs an intermediate orthonormal triad — the $\mathcal{D}$ frame, with basis vectors $\hat{d}_1, \hat{d}_2, \hat{d}_3$ — whose second basis vector is the thrust direction. The $\mathcal{D}$ frame is built twice, once in body coordinates and once in inertial coordinates, and the rotation that maps between the two representations is the commanded attitude $[\mathcal{RN}]$.

Specifically, the body thrust direction ${}^{\mathcal{B}}\hat{t}$ (read from `bodyHeadingInMsg`) is matched exactly to the inertial thrust reference ${}^{\mathcal{N}}\hat{t}_{\text{ref}}$ (a configuration parameter). The Sun direction ${}^{\mathcal{B}}\hat{r}_{S/B}$ (from `attNavInMsg`) and the solar array drive axis ${}^{\mathcal{B}}\hat{a}$ (configuration parameter) are used to construct the auxiliary triad axes that fix the spin about the thrust direction. The goal is to place the solar array drive axis orthogonal to the Sun direction with the remaining rotational degree of freedom. This constraint can only be perfectly met when the solar array drive axis is orthogonal to the thrust body axis. The triad frame is illustrated below.

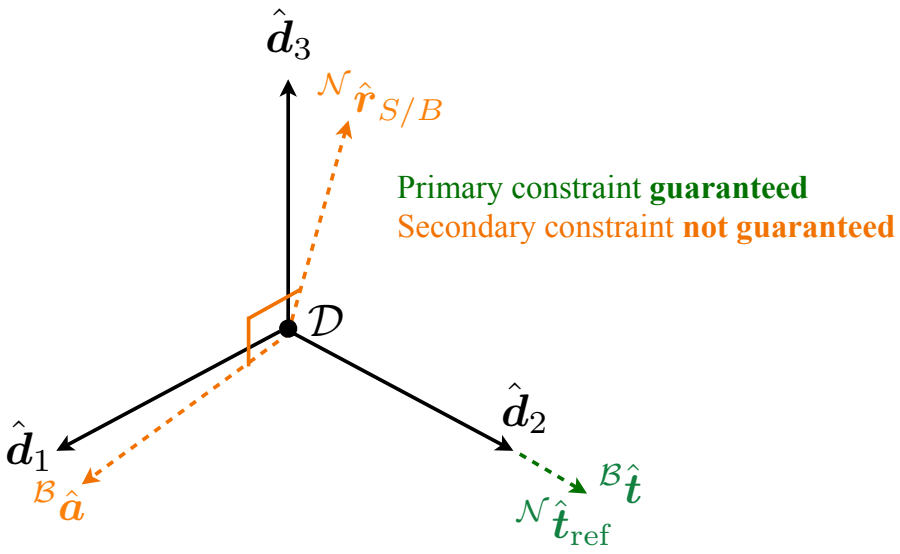

Figure 1: Illustration of the TRIAD reference frame construction

The mathematical details can be found in R. Calaon's PhD thesis, "Guidance, Control and Momentum Management of Spacecraft with Multiple Pointing Constraints". This is a single-precision (float32) port of the original double-precision Xmera implementation.

Module Architecture

The module is split into two layers:

- The **adapter** (`triad.h/.cpp`) is the SysModel-derived class that handles message I/O, validates configuration, builds an immutable `TriadConfig` from public properties, and constructs the algorithm via two-phase initialization.
- The **algorithm** (`triadAlgorithm.h/.cpp`) is a pure C++23 class with no framework dependencies. It takes message payloads as input, computes the attitude reference, and returns a payload struct as output. It must not throw from `update()`.

A pure-C shim (`triadAlgorithm_c.h/.cpp`) wraps the algorithm class for use by Ada/Adamant components via `extern "C"` bindings.

Adapter Layer

The adapter consumes the following messages and public configuration properties:

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
attNavInMsg	`NavAttMsgF32Payload`	Navigation attitude input (uses `sigma_BN` and `vehSunPntBdy` as ${}^{\mathcal{B}}\hat{r}_{S/B}$)
bodyHeadingInMsg	`BodyHeadingMsgF32Payload`	Body thrust direction input (uses `rHat_XB_B` as ${}^{\mathcal{B}}\hat{t}$)
attRefOutMsg	`AttRefMsgF32Payload`	Commanded attitude reference output (writes `sigma_RN`)

Table 2: Module Configuration Properties

Parameter Name	Type	Units	Default	Description	Bounds
sadaHat_B (required)	Eigen::Vector3f	[-]	zero	Solar array drive axis in body-frame coordinates ${}^{\mathcal{B}}\hat{a}$	Must be a unit vector (norm within 10^{-3} of 1); normalized on construction
thrustReqHat_N (required)	Eigen::Vector3f	[-]	zero	Inertial thrust reference direction ${}^{\mathcal{N}}\hat{t}_{\text{ref}}$	Must be a unit vector (norm within 10^{-3} of 1); normalized on construction
signOfZHat_N	float	[-]	+1	Sign of the inertial Z-axis used in fallback computation of the first triad axis when the Sun direction and the inertial thrust reference are nearly parallel	Must be non-zero; stored as sign$(\cdot)$ (± 1)

Algorithm Layer

Mathematical Formulation

Using the input messages and configuration parameters, the TRIAD algorithm constructs the commanded spacecraft attitude $[\mathcal{R}/\mathcal{N}]$ by building two orthonormal triad $\mathcal{D}$ frames — one in body frame $\mathcal{B}$ coordinates and one in inertial frame $\mathcal{N}$ coordinates.

Edge Case Guard 1

Before constructing the triad frames, the algorithm first checks for three unallowable edge cases. If any case is satisfied, the algorithm returns the current spacecraft attitude $\sigma_{\mathcal{B}/\mathcal{N}}$.

- Edge Case 1: Zero incoming body thrust direction ${}^{\mathcal{B}}\hat{t}$.
- Edge Case 2: Zero incoming Sun direction ${}^{\mathcal{B}}\hat{r}_{S/B}$.

- Edge Case 3: Solar array drive axis ${}^{\mathcal{B}}\hat{a}$ and incoming thrust direction ${}^{\mathcal{B}}\hat{t}$ are nearly parallel.

  Triggered when the angle between the solar array drive axis and the body thrust direction is less than a 5 degree threshold `kParallelThresholdRad`.

  $$\theta = \arccos\left(\left|{}^{\mathcal{B}}\hat{a} \cdot {}^{\mathcal{B}}\hat{t}\right|\right)$$

  Important

  Note that a fundamental assumption of the algorithm is that the thrust vector operates primarily in the spacecraft body Y-Z plane, making alignment between these vectors physically impossible.

Triad Frame in Body Coordinates

The first triad frame is expressed in spacecraft body frame $\mathcal{B}$ components. The frame is built from the body thrust direction and the solar array drive axis:

$$\begin{aligned}
{}^{\mathcal{B}}\hat{d}_2 &= {}^{\mathcal{B}}\hat{t} \\
{}^{\mathcal{B}}\hat{d}_3 &= {}^{\mathcal{B}}\hat{a} \times {}^{\mathcal{B}}\hat{d}_2 \\
{}^{\mathcal{B}}\hat{d}_1 &= {}^{\mathcal{B}}\hat{d}_2 \times {}^{\mathcal{B}}\hat{d}_3
\end{aligned}$$

Each axis is normalized. The matrix $[\mathcal{BD}]$ is constructed using the triad frame axes as columns: $[\mathcal{BD}] = \left[{}^{\mathcal{B}}\hat{d}_1,\ {}^{\mathcal{B}}\hat{d}_2,\ {}^{\mathcal{B}}\hat{d}_3\right]$.

Triad Frame in Inertial Coordinates

The second triad frame is expressed in inertial frame $\mathcal{N}$ components. The frame is build from the reference inertial thrust direction and the inertial Sun direction:

$$\begin{aligned}
{}^{\mathcal{N}}\hat{d}_2 &= {}^{\mathcal{N}}\hat{t}_{\text{ref}} \\
{}^{\mathcal{N}}\hat{d}_1 &= {}^{\mathcal{N}}\hat{r}_{S/B} \times {}^{\mathcal{N}}\hat{d}_2 \\
{}^{\mathcal{N}}\hat{d}_3 &= {}^{\mathcal{N}}\hat{d}_1 \times {}^{\mathcal{N}}\hat{d}_2
\end{aligned}$$

Each axis is normalized. The matrix $[\mathcal{ND}]$ is constructed using the triad frame axes as columns: $[\mathcal{ND}] = \left[{}^{\mathcal{N}}\hat{d}_1,\ {}^{\mathcal{N}}\hat{d}_2,\ {}^{\mathcal{N}}\hat{d}_3\right]$.

Edge Case Guard 2

Before constructing the second triad frame, the algorithm checks whether the Sun direction and the thrust inertial reference direction are nearly parallel using the same 5 degree threshold `kParallelThresholdRad`.

$$\theta = \arccos\left(\left|{}^{\mathcal{N}}\hat{r}_{S/B} \cdot {}^{\mathcal{N}}\hat{t}_{\text{ref}}\right|\right)$$

The second triad frame is ill-defined when these vectors are parallel. In this case, we instead seek to align the first triad axis with the positive or negative inertial frame Z-axis, depending on the configured `signOfZHat_N` parameter. To do so, ${}^{\mathcal{N}}\hat{d}_3$ is instead computed as the cross product between the thrust reference and the inertial Z-axis. Crossing the second triad axis with the new third triad axis ensures the first triad axis will align as close to the inertial Z-axis as possible.

$$\begin{aligned}
{}^{\mathcal{N}}\hat{d}_3 &= \pm{}^{\mathcal{N}}\hat{z} \times {}^{\mathcal{N}}\hat{t}_{\text{ref}} \\
{}^{\mathcal{N}}\hat{d}_1 &= {}^{\mathcal{N}}\hat{d}_2 \times {}^{\mathcal{N}}\hat{d}_3
\end{aligned}$$

Important

Because the spacecraft trajectory never passes directly beneath the Sun, the new cross products should never be zero. However, in the case where the thrust reference and sun direction are both nearly parallel to the fallback Z-axis, the new cross products will be zero. The current spacecraft attitude is returned in this case because this is an impossible configuration.

Commanded Attitude

The commanded body frame reference attitude is simply the composition:

$$[RN] = [BD]\,[ND]^T$$

The DCM result is converted to an MRP $\sigma_{\mathcal{R}/\mathcal{N}}$ and returned.

Algorithm Assumptions and Limitations

- Requirement: The solar array drive axis and inertial thrust reference direction must be unit vectors (norm within 10^{-3} of 1) and are normalized on construction.
- Requirement: The sign of the fallback inertial Z-axis cannot be set to zero.
- Edge Case 1&2: If either the incoming Sun direction vector or body thrust direction are zero, the algorithm returns the current spacecraft attitude.
- Assumption: The body thrust vector operates primarily in the spacecraft body Y-Z plane, making alignment with the solar array drive axis (body X axis) physically impossible.
- Edge Case 3: If the solar array drive axis is near parallel to the body thrust direction, the algorithm returns the current spacecraft attitude.
- Edge Case 4: If the Sun direction and the thrust inertial reference direction are nearly parallel, the second triad frame is ill-defined. In this case, the first and third inertial triad axes are computed using a fallback configured inertial Z-axis direction. The current spacecraft attitude is returned in the case where the fallback Z-axis is also nearly parallel to both the Sun direction and the thrust inertial reference direction.
- All math uses single-precision (float32). Compared to the double-precision Xmera implementation, regression tolerances are relaxed to roughly 10^{-6}.

Test Description

The module is verified through regression tests that compare the algorithm results against an independent reference implementation. Setup tests are used for the `TriadConfig` validators and round-trip tests are used to check the set configuration variables are correctly returned. Property tests check that (1) the algorithm output is finite for valid inputs, (2) the thrust body axis aligns with the thrust inertial heading direction, and that (3) the output MRP norm is bounded by 1 for any inputs. Edge-case tests check that (1) the current spacecraft attitude is returned when the thrust direction or sun direction messages are zero, (2) The current spacecraft attitude is returned when the solar array drive axis is aligned with the body thrust direction, (3) when the Sun direction is aligned with the thrust inertial reference, the fallback inertial Z-axis is used, and (4) When the solar array drive axis is orthogonal to the body thrust direction, the array-Sun orthogonality constraint is met. Fuzz tests randomize the configuration and inputs over reasonable ranges.